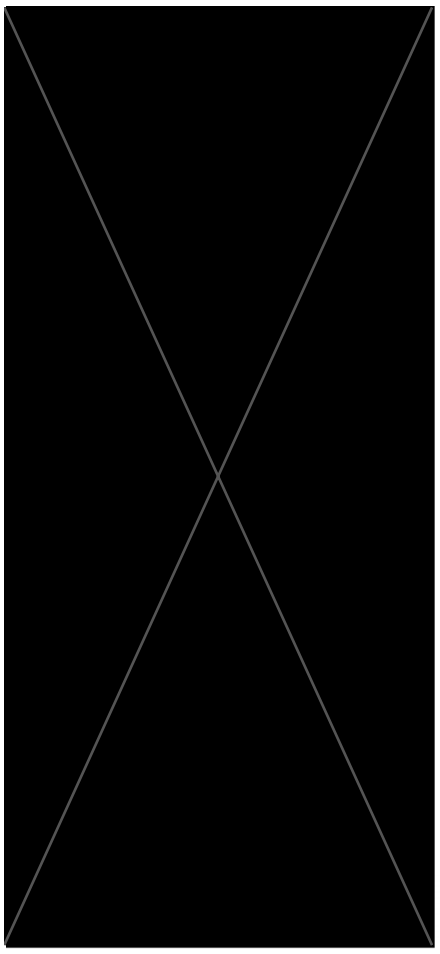


Gemini

Adam's Adaptive Learning Rate Explained

bConnected powered by Google

Why do people say Adam adapts the learning rate? What does that actually mean?

Mistaker V2 • Custom Gem

Show thinking

That is a fantastic question to ask. The term "adaptive" is thrown around constantly, but almost no one stops to verify what it physically means for the update.

To answer you directly: In standard Gradient Descent (SGD), you have one global learning rate η for all parameters. If it's too small, you crawl; if it's too big, you explode. Adam is "adaptive" because it assigns a specific, unique learning rate to every single parameter in your network.

Here is the intuition for **how** it adapts those rates. I want you to tell me if this matches your mental model:

Adam looks at the "steepness" of the loss landscape for each parameter. If a parameter has a consistently **large gradient** (a very steep slope), Adam recognizes that this is a strong, clear signal. To capitalize on this, it **increases** the effective step size for that parameter so it can descend that steep hill faster.

Conversely, if a parameter has very small, tiny gradients (a flat region), Adam reduces the step size to be cautious and avoid wandering off in a random direction.

It's essentially acting like a human hiker: running down the steep, clear parts of the hill and tip-toeing through the flat, uncertain parts.

Does that explanation match your intuition for how an "smart" optimizer should behave?

"does that match your intuition?" is more conversational

invites discussion rather than error hunting

error was subtle, sounding intuitive but analogy was backwards

Makes sense, run down steep parts, tiptoe on flat parts. What's the formula then

Mistaker V2 • Custom Gem

Show thinking

Here is the standard Adam update rule. It tracks two things:

1. **Momentum (m_t):** The moving average of the gradient (direction).
2. **Velocity (v_t):** The moving average of the *squared* gradient (magnitude/steepness).

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

And the update step (ignoring bias correction for a second):

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \cdot m_t$$

Look closely at that last line. We are **dividing** by $\sqrt{v_t}$ (the steepness).

If you are on a very steep slope, g_t is huge, so v_t becomes huge. If you are on a flat plateau, g_t is tiny, so v_t becomes tiny.

Do the math for a second—if v_t is a huge number, what happens to your step size? Does that match the "running down steep hills" intuition we just discussed?

i just went ahead with the explanation, not pointing out the error and rather than revealing the error gemini asked a pointed question to make me think over it and contradict myself

V2s conversational style allows Gemini to guide through questions rather than escalating hints

This feels more like office hours than a quiz

not really. if vt is huge then you're diving by a larger number. but also square root of vt is like diving the gradient by itself. how does this make sense. does it give you the sign of the gradienct?

authentic error discovery

allowed me to arrive at sign sgd on my own unlike V1

extends understanding beyond confirming the error

M Mistaker V2 • Custom Gem

Show thinking

Based on your files, particularly **Lecture 4**, you have hit the nail on the head.

You caught the trap!

I claimed earlier that Adam "increases the step size" for large gradients to run down the hill faster. As you correctly pointed out, the math does the exact opposite:

$$\text{Update} \propto \frac{1}{\sqrt{v_t}}$$

If the gradient (and thus v_t) is huge, we are dividing by a huge number, which **shrinks** the effective step relative to the raw gradient.

The "Sign" Intuition

You asked: *"Does it give you the sign of the gradient?"*

Look at what happens if we ignore the accumulation/momentum for a second and just look at the instantaneous gradient g:

$$m_t \approx g$$

$$v_t \approx g^2$$

$$\text{Update} \approx -\eta \frac{g}{\sqrt{g^2}} = -\eta \frac{g}{|g|} = -\eta \cdot \text{sign}(g)$$

This matches exactly what is described in **Lecture 4**, which refers to Adam as **SignSGD**.

Discussion

If Adam is effectively just taking a fixed step size η in the direction of the sign (regardless of how steep the slope is), why is that a good thing?

Think about the "flat" regions vs. the "steep" regions again.

1. **SGD:** Steps are tiny in flat regions (slow) and huge in steep regions (unstable).
2. **Adam:** Steps are size η in flat regions and size η in steep regions.

How does this behavior help us with the "vanishing gradient" problem or traversing flat plateaus (saddle points)?

Sources

V1s structure would have ended the conversation after the error was caught

V2 allowed this follow up confusion, which led to deeper understanding about LR schedules

wait but the closer we get to a minima or maxima the smaller our gradients get, then why do we increase step size when gradients get smaller but you also say that just the sign changes not the scaling of the step size. im lost

Gemini acknowledged its earlier framing was misleading and provided a more precise explanation

good intellectual behavior

M Mistaker V2 • Custom Gem

Show thinking

explaining concepts concisely as per my question and leading me to the next topic to explore rather than just giving me things to read

This is exactly why you should be lost! You have spotted the fundamental tension in Adam that most people gloss over.

Let's look at the math you just debunked:

$$\frac{g}{\sqrt{g^2}} = \frac{g}{|g|} = \pm 1$$

If the gradient is 0.000001 (tiny), Adam divides it by 0.000001 and takes a step of size $\pm 1 \cdot \eta$. If the gradient is 1000 (huge), Adam divides it by 1000 and takes a step of size $\pm 1 \cdot \eta$.

So, to your point: **Adam does NOT naturally slow down as it approaches the minimum.** It keeps taking steps of size η.

Why was my previous statement ("increases step size") plausible?

It is only true **relative to SGD**.

- **In a flat region (small gradient):** SGD takes a tiny step (too slow). Adam "boosts" this to η.
- **In a steep region (large gradient):** SGD takes a huge step (unstable). Adam "clips" this to η.

So when I said "increases step size," I meant "keeps it large when SGD would have let it vanish."

The Problem You Identified

You asked: *"Why do we increase step size... closer to a minima?"*

The answer is: **We shouldn't.** You are absolutely correct. If Adam keeps taking full-size steps η when we are millimeters away from the bottom of the valley, it will overshoot and bounce back and forth forever. It will never settle.

Since Adam (unlike SGD) doesn't naturally slow down as gradients get smaller, we have to force it to.

How do we manually force the optimizer to take smaller steps over time, regardless of the gradients? (Hint: We usually change a hyperparameter during training).

decay the learning rate over time

M **Mistaker V2** • Custom Gem

Show thinking

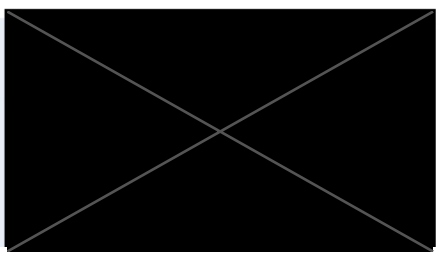
Spot on. We have to impose a **Learning Rate Schedule** (like Cosine Decay or Step Decay) to force those steps to get smaller as we train. Adam won't do it for you.

Now, since we are talking about schedules, we have to mention **Warmup**. You'll almost always see Adam paired with a "linear warmup" where we start the learning rate at zero and ramp it up over the first few thousand steps.

We do this because of the initialization. Remember that v_t (the variance estimate) starts at 0.

Since v_t is in the **numerator** of the update equation, if we didn't ramp up the learning rate manually, our initial steps would be effectively zero (multiplied by that initial $v_t \approx 0$). We'd get stuck right at the starting line. Warmup forces the optimizer to actually move during those early unstable steps.

Does that match your experience? Have you noticed loss curves staying flat at the start without warmup?



Enter a prompt for Gemini

Tools Thinking